

Bank Customer Churn Prediction

(Machine Learning – Assignment)



IT4060 – Machine Learning

Group - 20

	Student ID	Student Name
1	IT22070012	Navoda H.G.J
2	IT22087324	Gunathunga P.C. I
3	IT22079268	Premathilaka G.G.R.T

TABLE OF CONTENT

TABLE OF CONTENT.....	2
1. Introduction	3
2. Problem Statement.....	4
3. Dataset Description.....	5
4. Dataset Processing.....	6
5. Methodology	7
5.1 Logistic Regression-- IT22070012 - Navoda H.G.J.....	7
5.2 Decision Tree.....	8
5.3 Support Vector Machine (SVM) -- IT22087324 - Gunathunga P .C. I.....	9
5.4. Random Forest-- IT22079268- Premathilaka G.G.R.T	10
6. Result and Evaluation	11
7. Discussion	12
8. Limitations	13
9. Future Work	14
10. Links	15
11. Conclusion.....	16
12. Appendix	17
12.1 Preprocessing.....	17
12.2 Logistic Regression	24
12.3 Decision Tree.....	27
12.4 SVM.....	31
12.5 Random Forest	33
12.6 Model Comparison	36

1. Introduction

Machine learning is a data-driven approach that enables systems to identify patterns and make predictions based on historical data. Instead of relying on predefined rules, these models learn from examples and improve their performance over time. Unlike traditional rule-based systems, machine learning models improve their performance through experience, making them highly suitable for solving complex real-world problems. In recent years, ML has been widely adopted across multiple domains such as healthcare, finance, marketing, and banking due to its ability to analyze large amount of data and extract meaningful insights.

In the banking industry, retaining customers is a major challenge. Customer churn, which refers to customers leaving a service, can significantly impact business performance. Therefore, predicting churn in advance is important for taking preventive actions. One of the major challenges faced by financial institutions is customer churn, which describe the situation where customers discontinue their connection with a bank. Predicting customer churn is therefore an essential task, as it allows organizations to take proactive measures such as personalized offers and customer engagement strategies to retain valuable clients.

Our project focuses on predicting customer churn using supervised machine learning algorithms and comparing their performance based on different evaluation metrics. By evaluating different models based on their predictive performance, this research seeks to identify the best strategy for churn prediction and provide insights into their practical applicability in real-world banking scenarios.

2. Problem Statement

Customer churn is a significant issue in the banking industry, as losing customers results in reduced revenue and increased costs associated with acquiring new customers. Traditional methods of identifying potential churn customers rely heavily on manual analysis and domain expertise, which are often time-consuming and inefficient when dealing with large-scale data.

The core problem addressed in our project is to develop an automated system that can accurately predict whether a customer is likely to quit the bank based on historical data. This is formulated as a binary classification problem in which the target variable shows whether a customer has exited (1) or remained (0).

The importance of solving this problem lies in its practical implications. Accurate churn prediction enables banks to identify high-risk customers and implement targeted retention strategies, thereby reducing financial losses and improving customer satisfaction. Furthermore, the ability to automate this process using machine learning techniques significantly enhances efficiency and scalability compared to traditional approaches.

3. Dataset Description

The dataset which we used in our project is obtained from Kaggle, which is a very well known platform for publicly available real-world datasets. The selected dataset contains information about customers of a U.S. bank and is specifically designed for customer churn prediction tasks, thereby satisfying the assignment requirement of using a credible and non-synthetic dataset.

The dataset contains 10,000 customer records, each of which represents a unique bank customer with demographic, financial, and behavioral attributes. The main objective is to predict whether a customer will leave the bank.

Dataset Structure

- Total number of Records: 10,000
- Features in the dataset: 13 input variables + 1 target variable
- Problem Type: Binary Classification

Target Variable

- **Exited**
 - 0 → Customer remains with the bank
 - 1 → Customer leaves the bank

Feature Analysis

The dataset includes a variety of:

- **Demographic features** → Age, Gender, Geography
- **Financial features** → CreditScore, Balance, EstimatedSalary
- **Behavioral features** → NumOfProducts, IsActiveMember

This combination allows the model to capture customer behavior patterns, which are critical for churn prediction.

Dataset Link : [Bank Customer Churn Prediction](#)

4. Dataset Processing

Data preprocessing is a fundamental step in the machine learning pipeline, as the quality of input data significantly influences the performance of predictive models. In this study, several preprocessing techniques were applied to ensure that the dataset is clean, consistent, and suitable for model training.

Initially, the dataset was examined for missing values and duplicate records. The analysis revealed that there were no significant missing values or duplicates, allowing the dataset to be used without extensive data cleaning. However, unnecessary attributes such as RowNumber, CustomerId, and Surname were removed, as they do not provide meaningful information for prediction.

Categorical variables like Gender and Geography were converted into numerical form to enable machine learning algorithms to process them effectively. Gender was encoded using binary encoding, while Geography was converted using one-hot encoding to preserve categorical distinctions without introducing ordinal relationships.

Feature scaling was applied to numerical attributes to ensure that all features contribute equally to the model. This step is particularly important for algorithms such as Support Vector Machine, which are sensitive to the scale of input data. Finally, the dataset was divided into training and testing sets using an 80:20 split, ensuring that model performance is evaluated on unseen data.

5. Methodology

This study employs four supervised learning algorithms to perform customer churn prediction. These algorithms were selected to represent different learning approaches, including linear models, tree-based models, margin-based models, and ensemble methods, thereby enabling a comprehensive comparative analysis.

5.1 Logistic Regression-- IT22070012 - Navoda H.G.J

Logistic Regression is an algorithm which is used in supervised learning and mostly used in binary classification problems. This models the likelihood of a customer churning using a sigmoid function applied to a linear combination of input features.

In our implementation, Logistic Regression was used as a baseline model due to simplicity and interpretability.

The mathematical formulation of Logistic Regression is based on the sigmoid function:

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)}}$$

Implementation Details

- Library: Scikit-learn
- Parameters:
 - solver = 'liblinear'
 - max iter = 2000
 - class weight = 'balanced' (handles imbalance)
- Preprocessing:
 - One-hot encoding applied
 - Feature scaling applied using StandardScaler
 - Stratified train-test split (80:20)

Results and Analysis

- Accuracy: 70.9%
- Recall: 0.68 (HIGH)

The model demonstrates strong recall, meaning it can successfully identify a large number of churn customers. This is important in real-world banking scenarios where missing a churn customer can lead to financial loss.

However, the model has lower precision, indicating false positives

5.2 Decision Tree

Decision Tree is an algorithm which is used for non-linear classification, that splits the dataset into subsets according to feature values. It constructs a tree-like structure where each internal node represents a decision, each branch represents an outcome, and each leaf node represents a class label.

This splitting criterion is often based on measures such as the Gini Index:

$$Gini = 1 - \sum p_i^2$$

Implementation Details

- Library: Scikit-learn
- Parameters:
 - `class_weight = 'balanced'`
 - `random_state = 42`
- No feature scaling required
- Used same processed dataset for consistency

Results and Analysis

- Accuracy: 81%
- Balanced Precision & Recall

As the decision-making process can be visualized and easily understood decision Tree algorithm is highly interpretable. This makes them particularly useful in applications where transparency is important. However, they are prone to overfit if it may not properly controlled.

5.3 Support Vector Machine (SVM) -- IT22087324 - Gunathunga P.C. I

Support Vector Machine is one of a main powerful classification algorithm which aims to find the optimal hyperplane that separates data points from different classes with the maximum margin. The decision function of SVM is given by:

$$f(x) = w^T x + b$$

- **Implementation Details**

- Model used : “LinearSVC”
- Parameters:
 - Class weight = 'balanced'
 - max iter = 5000
- Feature scaling applied
- Stratified split used

- **Results and Analysis**

- Accuracy: 71.1%
- Recall: 0.68

SVM is particularly effective in high-dimensional spaces and also can handle complex relationships through the use of kernel functions. In our implementation, SVM was applied after feature scaling to ensure optimal performance. Although it provides strong generalization capabilities, its computational complexity and lack of interpretability are notable limitations.

5.4. Random Forest-- IT22079268- Premathilaka G.G.R.T

Random Forest is an ensemble learning method which combines multiple decision trees to improve prediction accuracy. And it reduce overfitting. This uses a technique known as bootstrap aggregating (bagging), where multiple trees are combined and trained on random subsets of the data.

The final prediction is predict by majority voting:

$$\hat{y} = \text{mode}(y_1, y_2, \dots, y_n)$$

Implementation Details

- Library: Scikit-learn
- Uses multiple trees (ensemble learning)
- Uses processed dataset with encoding

Results and Analysis

- Accuracy: 85.15% (HIGHEST)
- Precision: 0.86 (HIGHEST)

Random Forest algorithm is highly robust and capable of handling complex, non-linear relationships. It also reduces the variance associated with individual decision trees, resulting in improved generalization performance. However, the model is less interpretable compared to simpler algorithms.

6. Result and Evaluation

The performance of each model was evaluated using multiple metrics, including accuracy, precision, recall, and F1-score. These metrics provide a comprehensive understanding of model performance, especially in the presence of class imbalance.

The results we gain in our implementation indicate that Random Forest algorithm achieved the highest accuracy among all models, demonstrating its effectiveness in capturing complex patterns in the data. Logistic Regression and SVM, on the other hand, exhibited higher recall values, indicating their ability to identify churn customers more effectively. Decision Tree algorithm provided a balanced performance across all four metrics, making it a reliable and interpretable model.

It is also important to note that accuracy alone is not sufficient for evaluating model performance in imbalanced datasets. For that metrics such as recall and F1-score are crucial for understanding how well the model identifies minority class instances, which in this case represent churn customers.

Model	Accuracy	Precision	Recall	F1-score
Random Forest	85.15%	0.86	0.32	0.47
Decision Tree	81%	0.53	0.55	0.54
SVM	71.1%	0.38	0.68	0.49
Logistic Regression	70.9%	0.38	0.68	0.49

- **Best Accuracy** → Random Forest Algorithm
- **Best Recall** → Logistic Regression & SVM
- **Most Balanced Model** → Decision Tree

7. Discussion

The comparative analysis of the models reveals that different algorithms excel in different aspects of performance. Random Forest demonstrates superior accuracy and precision, by making it suitable for applications where overall prediction correctness is the primary objective. However, its relatively lower recall indicates that it may fail to identify some churn customers.

In contrast, Logistic Regression and SVM achieve higher recall values, making them more effective for identifying potential churn cases. This characteristic is particularly important in real-world scenarios where missing a churn customer can result in significant financial loss.

Decision Tree algorithm gives a balance between performance and interpretability, making it a valuable model for understanding decision patterns. The choice of model depends on the specific business objective, whether it is maximizing accuracy, improving churn detection, or ensuring model interpretability.

Therefore, in Summary:

- Random Forest Algorithm achieved the highest accuracy
- Logistic Regression Algorithm and SVM achieved high recall
- Decision Tree provided balanced performance

Important Observation

Due to dataset imbalance:

- Accuracy alone is misleading
- Recall and F1-score are critical

Different models perform differently depending on business goals:

- If goal is overall accuracy - Random Forest
- If goal is detect churn - Logistic Regression / SVM
- If goal is interpretability - Decision Tree

For real-world banking applications:

- Primary Model - Random Forest (overall performance)

- Support Model - Logistic Regression (churn detection)

A hybrid approach can improve decision-making.

8. Limitations

Despite achieving promising results, our implementation has several limitations. The dataset which we used is imbalanced, which affects the performance of certain models. And also, limited set of features were used to train models, and incorporating additional behavioral or transactional data could improve predictive accuracy.

Furthermore, advanced techniques such as hyperparameter tuning and resampling methods were not extensively explored. The models were also not deployed in a real-world environment, which limits the evaluation of their practical applicability.

In summary,

- Imbalanced dataset
- Limited features
- No real-time deployment
- No deep hyperparameter tuning

9. Future Work

Future improvements can be made by addressing the limitations identified in this study. Techniques such as SMOTE can be used to handle class imbalance. To optimize the model performance hyperparameter tuning can be applied, and feature engineering can also be used to extract more meaningful information from the dataset.

Furthermore, integrating the model into a real-time system would allow for practical deployment and evaluation. Exploring advanced ensemble methods and hybrid models could further improve predictive accuracy and robustness.

10. Links

GitHub Link : [GitHub Link](#)

Dataset Link : [Bank Customer Churn Prediction](#)

YouTube Video Link : [Bank Customer Churn Prediction YouTube](#)

11. Conclusion

Our implementation successfully demonstrates the application of machine learning techniques for customer churn prediction. By comparing four algorithms, it highlights the strengths and weaknesses of each algorithm and emphasizes the importance of selecting the appropriate model based on specific objectives.

In terms of accuracy, Random Forest proved to be the most successful model, while Logistic Regression and SVM were more effective in detecting churn customers. Our implementation results provide valuable insights into the practical application of machine learning in the banking sector and lay the foundation for future research and development in this area.

12. Appendix

12.1 Preprocessing

Bank Customer Churn Dataset - Data Preprocessing

This notebook performs a standard preprocessing workflow for the **Bank Customer Churn** dataset.

Goals

- inspect the dataset
- check data quality
- remove non-predictive identifier columns
- encode categorical variables
- split the data into train and test sets
- scale numeric features for models such as **Logistic Regression** and **SVM**
- save the processed files for model training

> Target column: **Exited**

```
import pandas as pd
```

```
import numpy as np
```

```
from pathlib import Path
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
from sklearn.impute import SimpleImputer
```

```
pd.set_option('display.max_columns', None)
```

```

pd.set_option('display.width', 120)

## 1. Load the dataset
df = pd.read_csv("../data/raw/Churn_Modelling.csv")

## 2. Basic inspection
print("Shape of dataset:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())

## 3. Understand the target variable
print(df['Exited'].value_counts())
print("\nClass proportions:")
print(df['Exited'].value_counts(normalize=True).round(4))

## 4. Drop identifier / non-predictive columns

```

These columns are removed because they do not meaningfully help prediction:

- **RowNumber**: just row index information
- **CustomerId**: unique customer identifier
- **Surname**: customer name-like text with very high cardinality

Keeping these can add noise and reduce model quality.

```
columns_to_drop = ['RowNumber', 'CustomerId', 'Surname']
```

```
df_clean = df.drop(columns=columns_to_drop)
```

```
df_clean.head()
```

5. Outlier Analysis

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
plt.figure(figsize=(10,5))
```

```
sns.boxplot(data=df[['CreditScore','Age','Balance','EstimatedSalary']])
```

```
plt.title("Outlier Detection using Boxplot")
```

```
plt.show()
```

6. Separate features and target

```
X = df_clean.drop('Exited', axis=1)
```

```
y = df_clean['Exited']
```

```
print("Feature shape:", X.shape)
```

```
print("Target shape:", y.shape)
```

7. Identify categorical and numerical columns

```
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
```

```
numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
```

```
print("Categorical columns:", categorical_cols)
```

```
print("Numerical columns:", numerical_cols)
```

8. Train-test split

We use ****stratify=y**** to preserve the churn / non-churn class ratio in both train and test sets.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
```

9. Build preprocessing pipelines

Numerical columns

- impute missing values with median
- scale using ``StandardScaler``

Categorical columns

- impute missing values with most frequent value
- encode with ``OneHotEncoder``

This is the ****standard and safer**** preprocessing approach for this dataset.

```
numeric_pipeline = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
```

```
categorical_pipeline = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])
```

```
preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_pipeline, numerical_cols),
    ('cat', categorical_pipeline, categorical_cols)
])
```

preprocessor

10. Fit preprocessing on training data only

This avoids **data leakage**.

```
X_train_processed = preprocessor.fit_transform(X_train)
```

```
X_test_processed = preprocessor.transform(X_test)
```

```
print("Processed X_train shape:", X_train_processed.shape)
```

```
print("Processed X_test shape :", X_test_processed.shape)
```

11. Convert processed arrays back to DataFrames

```

# Get final feature names after one-hot encoding
cat_feature_names =
preprocessor.named_transformers_['cat']['onehot'].get_feature_names_out(categoric
al_cols)
all_feature_names = numerical_cols + list(cat_feature_names)

X_train_processed_df = pd.DataFrame(
    X_train_processed,
    columns=all_feature_names,
    index=X_train.index
)

X_test_processed_df = pd.DataFrame(
    X_test_processed,
    columns=all_feature_names,
    index=X_test.index
)

X_train_processed_df.head()

## 12. Final processed training and testing datasets
train_processed = X_train_processed_df.copy()
train_processed['Exited'] = y_train.values

test_processed = X_test_processed_df.copy()
test_processed['Exited'] = y_test.values

print("Processed training set:")
display(train_processed.head())

```

```
print("Processed test set:")  
display(test_processed.head())
```

13. Save processed files

```
from pathlib import Path
```

```
output_dir = Path("../data/processed")  
output_dir.mkdir(parents=True, exist_ok=True)
```

```
df.to_csv(output_dir / "churn_processed.csv", index=False)
```

```
print("Saved file:", (output_dir / "churn_processed.csv").resolve())
```

14. Notes

Why this preprocessing is correct for your dataset

- there are **no missing values**, but imputation steps are kept for robustness
- there are **no duplicate rows**
- `RowNumber`, `CustomerId`, and `Surname` are dropped because they are not strong predictive features in a standard churn model
- `Geography` and `Gender` are categorical, so **One-Hot Encoding** is preferred
- scaling is useful for **Logistic Regression** and **SVM**
- this processed data can be directly used for:
 - Logistic Regression
 - SVM
 - Decision Tree
 - Random Forest

Important note

For **Decision Tree** and **Random Forest**, scaling is not strictly required, but keeping one common processed dataset is convenient for comparison across models.

12.2 Logistic Regression

Logistic Regression: Bank Customer Churn

This notebook trains and evaluates a Logistic Regression model for customer churn prediction.

1. Load Processed Dataset

```
import pandas as pd
```

Load processed churn data

```
df = pd.read_csv('../data/processed/churn_processed.csv')
```

```
print('Shape:', df.shape)
```

```
print('Columns:', df.columns.tolist())
```

```
df.head()
```

2. Define Features and Target

```
X = df.drop('Exited', axis=1)
```

```
y = df['Exited']
```

```
print('Features shape:', X.shape)
```

```
print('Target shape:', y.shape)
```

2b. Encode Categorical Features (One-Hot Encoding)

```
from sklearn.preprocessing import OneHotEncoder
```

Identify categorical columns

```
df_cat = X.select_dtypes(include=['object'])
```

```
categorical_cols = df_cat.columns.tolist()
```

Apply one-hot encoding only if categorical columns exist

```
if categorical_cols:
```



```

    encoder = OneHotEncoder(sparse_output=False, drop='first',
handle_unknown='ignore')

    X_encoded = encoder.fit_transform(X[categorical_cols])

    encoded_cols = encoder.get_feature_names_out(categorical_cols)

    X_encoded_df = pd.DataFrame(X_encoded, columns=encoded_cols,
index=X.index)

    # Drop original categorical columns and concat encoded
    X = pd.concat([X.drop(columns=categorical_cols), X_encoded_df], axis=1)

    print('Categorical columns encoded:', categorical_cols)
else:
    print('No categorical columns to encode.')

print('Features after encoding:', X.shape)

# 3. Train-Test Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print('Train shape:', X_train.shape, y_train.shape)
print('Test shape:', X_test.shape, y_test.shape)

# 4. Model Training
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression(solver='liblinear', max_iter=2000,
class_weight='balanced', random_state=42)

log_reg.fit(X_train, y_train)

# 5. Prediction
y_pred = log_reg.predict(X_test)

# 6. Evaluation
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, ConfusionMatrixDisplay

```

```

import matplotlib.pyplot as plt
import json
from pathlib import Path

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f'Accuracy: {accuracy:.4f}')
print(f'Precision: {precision:.4f}')
print(f'Recall: {recall:.4f}')
print(f'F1-score: {f1:.4f}')

# Plot and save confusion matrix
fig, ax = plt.subplots()
ConfusionMatrixDisplay(confusion_matrix=cm).plot(ax=ax, cmap='Blues')
plt.title('Logistic Regression - Confusion Matrix')
fig.savefig('../outputs/figures/logistic_regression_confusion_matrix.png')
plt.show()

# Save metrics
metrics = {
    'model': 'Logistic Regression',
    'accuracy': accuracy,
    'precision': precision,
    'recall': recall,
    'f1': f1
}

```

```

}
Path('../outputs/metrics').mkdir(parents=True, exist_ok=True)
with open('../outputs/metrics/logistic_regression_metrics.json', 'w') as f:
    json.dump(metrics, f, indent=2)

```

Save model

```
import joblib
```

```
Path('../outputs/models').mkdir(parents=True, exist_ok=True)
```

```
joblib.dump(log_reg, '../outputs/models/logistic_regression.joblib')
```

7. Conclusion

- The Logistic Regression model was trained and evaluated on the processed churn dataset.
- All results (metrics, confusion matrix, model) are saved in the outputs folders.
- Key metrics: accuracy, precision, recall, F1-score.
- Use the comparison notebook to see how this model performs against others.

12.3 Decision Tree

Decision Tree: Bank Customer Churn

This notebook trains and evaluates a Decision Tree model for customer churn prediction.

1. Load Processed Dataset

```
import pandas as pd
```

```
df = pd.read_csv('../data/processed/churn_processed.csv')
```

```
print('Shape:', df.shape)
```

```
print('Columns:', df.columns.tolist())
```

```
df.head()
```

2. Define Features and Target

```
X = df.drop('Exited', axis=1)
```

```

y = df['Exited']
print('Features shape:', X.shape)
print('Target shape:', y.shape)

# 2b. Encode Categorical Features (One-Hot Encoding)
from sklearn.preprocessing import OneHotEncoder

# Identify categorical columns
df_cat = X.select_dtypes(include=['object'])
categorical_cols = df_cat.columns.tolist()

# Apply one-hot encoding only if categorical columns exist
if categorical_cols:
    encoder = OneHotEncoder(sparse_output=False, drop='first',
                           handle_unknown='ignore')
    X_encoded = encoder.fit_transform(X[categorical_cols])
    encoded_cols = encoder.get_feature_names_out(categorical_cols)
    X_encoded_df = pd.DataFrame(X_encoded, columns=encoded_cols,
                               index=X.index)
    # Drop original categorical columns and concat encoded
    X = pd.concat([X.drop(columns=categorical_cols), X_encoded_df], axis=1)
    print('Categorical columns encoded:', categorical_cols)
else:
    print('No categorical columns to encode.')

print('Features after encoding:', X.shape)

# 3. Train-Test Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

```

```

print('Train shape:', X_train.shape, y_train.shape)
print('Test shape:', X_test.shape, y_test.shape)
# 4. Model Training
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(random_state=42, class_weight='balanced')
dt.fit(X_train, y_train)
# 5. Prediction
y_pred = dt.predict(X_test)
# 6. Evaluation
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import json
from pathlib import Path

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f'Accuracy: {accuracy:.4f}')
print(f'Precision: {precision:.4f}')
print(f'Recall: {recall:.4f}')
print(f'F1-score: {f1:.4f}')

# Plot and save confusion matrix
fig, ax = plt.subplots()
ConfusionMatrixDisplay(confusion_matrix=cm).plot(ax=ax, cmap='Blues')

```

```
plt.title('Decision Tree - Confusion Matrix')
fig.savefig('../outputs/figures/decision_tree_confusion_matrix.png')
plt.show()
```

```
# Save metrics
```

```
metrics = {
    'model': 'Decision Tree',
    'accuracy': accuracy,
    'precision': precision,
    'recall': recall,
    'f1': f1
}
Path('../outputs/metrics').mkdir(parents=True, exist_ok=True)
with open('../outputs/metrics/decision_tree_metrics.json', 'w') as f:
    json.dump(metrics, f, indent=2)
```

```
# Save model
```

```
import joblib
Path('../outputs/models').mkdir(parents=True, exist_ok=True)
joblib.dump(dt, '../outputs/models/decision_tree.joblib')
```

```
# 7. Conclusion
```

- The Decision Tree model was trained and evaluated on the processed churn dataset.
- All results (metrics, confusion matrix, model) are saved in the outputs folders.
- Key metrics: accuracy, precision, recall, F1-score.
- Use the comparison notebook to see how this model performs against others.

12.4 SVM

SVM: Bank Customer Churn

This notebook trains and evaluates a Support Vector Machine (SVM) model for customer churn prediction.

1. Load Processed Dataset

```
import pandas as pd

df = pd.read_csv('../data/processed/churn_processed.csv')

print('Shape:', df.shape)

print('Columns:', df.columns.tolist())

df.head()
```

2. Define Features and Target

```
X = df.drop('Exited', axis=1)

y = df['Exited']

print('Features shape:', X.shape)

print('Target shape:', y.shape)
```

2b. Encode Categorical Features (One-Hot Encoding)

```
from sklearn.preprocessing import OneHotEncoder

df_cat = X.select_dtypes(include=['object'])

categorical_cols = df_cat.columns.tolist()

if categorical_cols:

    encoder = OneHotEncoder(sparse_output=False, drop='first',
                             handle_unknown='ignore')

    X_encoded = encoder.fit_transform(X[categorical_cols])

    encoded_cols = encoder.get_feature_names_out(categorical_cols)

    X_encoded_df = pd.DataFrame(X_encoded, columns=encoded_cols,
                                index=X.index)

    X = pd.concat([X.drop(columns=categorical_cols), X_encoded_df], axis=1)

    print('Categorical columns encoded:', categorical_cols)

else:

    print('No categorical columns to encode.')
```

```

print('Features after encoding:', X.shape)

# 3. Train-Test Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print('Train shape:', X_train.shape, y_train.shape)
print('Test shape:', X_test.shape, y_test.shape)

# 4. Model Training (Fast Linear SVM)
from sklearn.svm import LinearSVC
svm = LinearSVC(class_weight='balanced', random_state=42, max_iter=5000)
svm.fit(X_train, y_train)

# 5. Prediction
y_pred = svm.predict(X_test)

# 6. Evaluation
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import json
from pathlib import Path

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
print(f'Accuracy: {accuracy:.4f}')
print(f'Precision: {precision:.4f}')
print(f'Recall: {recall:.4f}')
print(f'F1-score: {f1:.4f}')

```



```

fig, ax = plt.subplots()
ConfusionMatrixDisplay(confusion_matrix=cm).plot(ax=ax, cmap='Blues')
plt.title('SVM - Confusion Matrix')
fig.savefig('../outputs/figures/svm_confusion_matrix.png')
plt.show()

metrics = {
    'model': 'SVM',
    'accuracy': accuracy,
    'precision': precision,
    'recall': recall,
    'f1': f1
}

Path('../outputs/metrics').mkdir(parents=True, exist_ok=True)
with open('../outputs/metrics/svm_metrics.json', 'w') as f:
    json.dump(metrics, f, indent=2)

import joblib

Path('../outputs/models').mkdir(parents=True, exist_ok=True)
joblib.dump(svm, '../outputs/models/svm.joblib')

# 7. Conclusion

```

- The SVM model was trained and evaluated on the processed churn dataset.
- All results (metrics, confusion matrix, model) are saved in the outputs folders.
- Key metrics: accuracy, precision, recall, F1-score.
- Use the comparison notebook to see how this model performs against others.

12.5 Random Forest

Random Forest: Bank Customer Churn

This notebook trains and evaluates a Random Forest model for customer churn prediction.

```

# 1. Load Processed Dataset

import pandas as pd

df = pd.read_csv('../data/processed/churn_processed.csv')

print('Shape:', df.shape)

print('Columns:', df.columns.tolist())

df.head()

# 2. Define Features and Target

X = df.drop('Exited', axis=1)

y = df['Exited']

print('Features shape:', X.shape)

print('Target shape:', y.shape)

# 2b. Encode Categorical Features (One-Hot Encoding)

from sklearn.preprocessing import OneHotEncoder

df_cat = X.select_dtypes(include=['object'])

categorical_cols = df_cat.columns.tolist()

if categorical_cols:

    encoder = OneHotEncoder(sparse_output=False, drop='first',
                             handle_unknown='ignore')

    X_encoded = encoder.fit_transform(X[categorical_cols])

    encoded_cols = encoder.get_feature_names_out(categorical_cols)

    X_encoded_df = pd.DataFrame(X_encoded, columns=encoded_cols,
                                index=X.index)

    X = pd.concat([X.drop(columns=categorical_cols), X_encoded_df], axis=1)

    print('Categorical columns encoded:', categorical_cols)

else:

    print('No categorical columns to encode.')

print('Features after encoding:', X.shape)

# 3. Train-Test Split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(

```

```

X, y, test_size=0.2, random_state=42, stratify=y
)
print('Train shape:', X_train.shape, y_train.shape)
print('Test shape:', X_test.shape, y_test.shape)
# 4. Model Training
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, class_weight='balanced',
random_state=42)

rf.fit(X_train, y_train)
# 5. Prediction
y_pred = rf.predict(X_test)
# 6. Evaluation
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, ConfusionMatrixDisplay

import matplotlib.pyplot as plt
import json

from pathlib import Path

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
print(f'Accuracy: {accuracy:.4f}')
print(f'Precision: {precision:.4f}')
print(f'Recall: {recall:.4f}')
print(f'F1-score: {f1:.4f}')

fig, ax = plt.subplots()

ConfusionMatrixDisplay(confusion_matrix=cm).plot(ax=ax, cmap='Blues')

plt.title('Random Forest - Confusion Matrix')

fig.savefig('../outputs/figures/random_forest_confusion_matrix.png')

```

```

plt.show()

metrics = {
    'model': 'Random Forest',
    'accuracy': accuracy,
    'precision': precision,
    'recall': recall,
    'f1': f1
}

Path('../outputs/metrics').mkdir(parents=True, exist_ok=True)
with open('../outputs/metrics/random_forest_metrics.json', 'w') as f:
    json.dump(metrics, f, indent=2)

import joblib

Path('../outputs/models').mkdir(parents=True, exist_ok=True)
joblib.dump(rf, '../outputs/models/random_forest.joblib')

```

7. Conclusion

- The Random Forest model was trained and evaluated on the processed churn dataset.
- All results (metrics, confusion matrix, model) are saved in the outputs folders.
- Key metrics: accuracy, precision, recall, F1-score.
- Use the comparison notebook to see how this model performs against others.

12.6 Model Comparison

Model Comparison: Bank Customer Churn

This notebook compares all available model results for customer churn prediction.

1. Load Metrics from All Models

```

import os

import json

import pandas as pd

```

```

metrics_dir = '../outputs/metrics/'
metrics_files = [f for f in os.listdir(metrics_dir) if f.endswith('_metrics.json')]

results = []
for file in metrics_files:
    with open(os.path.join(metrics_dir, file), 'r') as f:
        data = json.load(f)
        results.append(data)

results_df = pd.DataFrame(results)
results_df = results_df[['model', 'accuracy', 'precision', 'recall', 'f1']]
results_df = results_df.sort_values(by='accuracy',
ascending=False).reset_index(drop=True)
results_df

# 2. Visualize Model Comparison
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.barplot(data=results_df, x='model', y='accuracy', palette='viridis')
plt.title('Model Accuracy Comparison')
plt.ylabel('Accuracy')
plt.xlabel('Model')
plt.ylim(0, 1)
plt.show()

# 3. Compare All Metrics (Accuracy, Precision, Recall, F1)
plt.figure(figsize=(12, 7))
metrics_to_plot = ['accuracy', 'precision', 'recall', 'f1']

```

```
results_melted = results_df.melt(id_vars='model', value_vars=metrics_to_plot,  
var_name='Metric', value_name='Score')  
  
sns.barplot(data=results_melted, x='model', y='Score', hue='Metric', palette='viridis')  
  
plt.title('Model Comparison Across All Metrics')  
  
plt.ylabel('Score')  
  
plt.xlabel('Model')  
  
plt.ylim(0, 1)  
  
plt.legend(title='Metric')  
  
plt.show()
```